UNIVERSIDAD PRIVADA FRANZ TAMAYO

SEDE EL ALTO

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS

UNIT TESTING DE LAS FUNCIONES CRÍTICAS DEL SISTEMA DE PRODUCTOS Y VENTAS

AUTOR: BRAYAN QUISPE MAMANI

ASIGNATURA: TESTING DE SOFTWARE

DOCENTE:

NELSON CRISTHIAN URUCHI QUISPE



EL ALTO – BOLIVIA

2026

Instrucción de la tarea y rúbrica de evaluación....1
1. Introducción....1
1.1 Objetivo....2
1.2 Herramientas utilizadas....2
2. Justificación técnica de la criticidad....2
2.1 Matriz de criticidad....2
2.2 VentaRepository.registrar....3
2.3 app.iniciar_sesion....3
2.4 app.requiere_rol....4
2.5 validaciones.validar_precio y validar_cantidad....4
2.6 Funciones de menor criticidad descartadas....4
3. Código de las funciones a probar....4
3.1 validaciones.validar_precio....4
3.2 validaciones.validar_cantidad....5
3.3 models.VentaRepository.registrar....5
3.4 app.iniciar_sesion....6
3.5 app.requiere_rol....6
4. Código de los Unit Tests....7
4.1 Pruebas de validar_precio (5 casos)....7
4.2 Pruebas de validar_cantidad (5 casos)....8
4.3 Pruebas de VentaRepository.registrar (5 casos)....8
4.4 Pruebas de iniciar_sesion (5 casos)....10
4.5 Pruebas de requiere_rol (5 casos)....12
5. Casos de prueba: Arrange, Act y Assert....13
5.1 Casos de validar_precio....13

5.2 Casos de validar_cantidad 14
5.3 Casos de VentaRepository.registrar 14
5.4 Casos de iniciar_sesion 14
5.5 Casos de requiere_rol 15
6. Evidencia de ejecución 15
7. Conclusiones 18
BIBLIOGRAFÍA 18

Instrucción de la tarea y rúbrica de evaluación

Analizar y elaborar los Unit Test de 5 funciones (las más críticas) dentro del sistema que está desarrollando para su proyecto integrador. Para cada caso, debe presentar: Arrange, Act y Assert, el código de su función o método, el código de sus pruebas y sus respectivas capturas.

- Modalidad: individual.
- Formato de presentación: documento en PDF, describiendo claramente cada punto instruido.
- Retroalimentación: en plataforma.
- Recuerde que se penaliza (5% por día) la entrega con retraso.

Tabla A. Correspondencia entre la rúbrica y las secciones del informe.

Criterio evaluado	Peso	Sección donde se cumple
1. Justifica técnicamente la criticidad de sus funciones frente a otras de menor criticidad.	20%	Sección 2
2. Comparte el código de las funciones a probar.	20%	Sección 3
3. Comparte el código de sus Unit Test, con Assert.	20%	Sección 4
4. Identifica los casos de prueba (Arrange), obtiene sus respuestas (Act) y realiza la comparación (Assert).	20%	Sección 5
5. Demuestra el funcionamiento de sus pruebas mostrando capturas.	15%	Sección 6
6. Documento ordenado, claramente identificable en cada punto y con ortografía impecable.	5%	Todo el documento

1. Introducción

El presente informe documenta el diseño y la ejecución de pruebas unitarias sobre las cinco funciones más críticas del sistema de gestión de productos y ventas desarrollado como proyecto integrador. El sistema consiste en una API REST construida con Flask que administra productos, compradores, usuarios y ventas, con autenticación por roles y control de inventario.

Una prueba unitaria aísla la unidad más pequeña de código (en este caso, una función o un método) y comprueba su comportamiento con entradas controladas, sin depender de la base de datos ni de servicios externos. Para ello se aplicó el patrón Arrange–Act–Assert: primero se prepara el escenario, luego se ejecuta la acción y finalmente se compara el resultado obtenido con el esperado.

1.1 Objetivo

Analizar y elaborar las pruebas unitarias de las cinco funciones más críticas del sistema, justificando técnicamente su criticidad, presentando el código bajo prueba, el código de las pruebas y la evidencia de su ejecución satisfactoria.

1.2 Herramientas utilizadas

- Lenguaje: Python 3.14.
- Marco de pruebas: pytest (versión 9.1.1).
- Componentes simulados: dobles de prueba (fakes) y monkeypatch para aislar la base de datos.
- Cliente de pruebas de Flask (test_client) para las funciones de la capa web.
- Entorno: sistema operativo Linux.

2. Justificación técnica de la criticidad

No todas las funciones del sistema merecen el mismo esfuerzo de prueba. La criticidad de una función se evaluó con cuatro criterios técnicos: el impacto de un fallo (qué se pierde si la función se comporta mal), la frecuencia de uso (cuántas veces se ejecuta en la operación real), el acoplamiento (cuántas otras partes dependen de ella) y la dificultad de detección del error (si el fallo se manifiesta de inmediato o permanece oculto). Las cinco funciones seleccionadas concentran el mayor producto de estos cuatro criterios.

2.1 Matriz de criticidad

Tabla 1. Matriz de criticidad de las funciones analizadas.

Función	Impacto del	Frecuencia	Detección	Criticidad

	fallo			
VentaRepository.registrar	Pérdida de dinero e inventario inconsistente	Muy alta	Difícil	Muy alta
app.iniciar_sesion	Acceso no autorizado y suplantación de identidad	Muy alta	Media	Muy alta
app.requiere_rol	Escalada de privilegios entre roles	Alta	Media	Muy alta
validaciones.validar_precio	Precios inválidos y totales de venta erróneos	Alta	Fácil	Alta
validaciones.validar_cantidad	Sobregiro de stock y ventas nulas o negativas	Alta	Fácil	Alta

2.2 VentaRepository.registrar

Es la función más crítica del sistema porque concentra la lógica de negocio en una sola transacción: bloquea al comprador y al producto, valida el stock disponible, descuenta el inventario, calcula el total con aritmética decimal y registra la venta. Si falla, o se cobra un monto incorrecto o se descuenta stock sin dejar constancia de la venta. Además, su acoplamiento es máximo: depende de las tablas de productos, compradores, usuarios y ventas, y su error no aparece de inmediato, sino cuando el inventario ya no cuadra.

2.3 app.iniciar_sesion

Es la puerta de entrada al sistema. Verifica que el usuario exista, que la cuenta esté activa y que la contraseña coincida con su hash antes de crear la sesión. Un fallo aquí permitiría el ingreso con credenciales incorrectas o la suplantación de otro rol, por lo que su impacto es de seguridad y afecta a todos los módulos posteriores.

2.4 app.requiere_rol

Es el control de autorización del lado del servidor. Envuelve cada endpoint protegido y decide si el usuario autenticado puede ejecutar la operación. Su criticidad es transversal: si el decorador falla, un vendedor podría eliminar productos, crear usuarios o ver información reservada al administrador. La validación en el cliente no sustituye este control, de modo que es la última línea de defensa.

2.5 validaciones.validar_precio y validar_cantidad

Ambas funciones son la primera barrera de integridad de datos. validar_precio impide que un producto se registre con precio nulo, negativo, infinito o desmesurado, y valida_cantidad impide vender cero, negativos o cantidades que desbordan el inventario. Su detección es más sencilla, por eso su criticidad se clasifica como alta y no como muy alta, pero se incluyen porque alimentan directamente a las tres funciones anteriores.

2.6 Funciones de menor criticidad descartadas

Quedaron fuera del alcance funciones como index, serializar, datos_publicos, los métodos listar y obtener de los repositorios, y validar_descripcion. Se trata de funciones de presentación o de simple consulta: no transforman datos financieros ni decisiones de seguridad, y un error en ellas produce un resultado visible de inmediato que no compromete la integridad del sistema. Esa diferencia de impacto es, precisamente, la que justifica concentrar las pruebas unitarias en las cinco funciones seleccionadas.

3. Código de las funciones a probar

A continuación se presenta el código fuente de cada función bajo prueba, extraído de los archivos validaciones.py, models.py y app.py del proyecto.

3.1 validaciones.validar_precio

```
def validar_precio(precio):
    """El precio debe ser numérico y mayor a 0."""
    if precio is None:
        raise ValueError("El precio no puede ser nulo")
    try:
```

```
        precio = float(precio)
    except (ValueError, TypeError):
        raise ValueError("El precio debe ser un número")
    if not math.isfinite(precio):
        raise ValueError("El precio debe ser un número finito")
    if precio <= 0:
        raise ValueError("El precio debe ser mayor a 0")
    if precio > 99999999.99:
        raise ValueError("El precio es demasiado alto")
    return round(precio, 2)
```

3.2 validaciones.validar_cantidad

```
def validar_cantidad(cantidad, permitir_cero=False):
    if cantidad is None or isinstance(cantidad, bool):
        raise ValueError("La cantidad debe ser un número")
    try:
        cantidad = float(cantidad)
    except (ValueError, TypeError):
        raise ValueError("La cantidad debe ser un número")
    if not math.isfinite(cantidad):
        raise ValueError("La cantidad debe ser un número finito")
    minimo = 0 if permitir_cero else 0.001
    if cantidad < minimo:
        mensaje = "La cantidad no puede ser negativa" if permitir_cero else "La cantidad
debe ser mayor a 0"
        raise ValueError(mensaje)
    if cantidad > 999999999.999:
        raise ValueError("La cantidad es demasiado alta")
    return round(cantidad, 3)
```

3.3 models.VentaRepository.registrar

```
    def registrar(self, producto_id, vendedor_id, cantidad, comprador_id):
        conn = get_connection()
        cursor = conn.cursor(dictionary=True)
        try:
            conn.start_transaction()
            cursor.execute(
                "SELECT id FROM compradores WHERE id = %s FOR UPDATE",
                (comprador_id,)
            )
            if cursor.fetchone() is None:
                raise CompradorNoEncontrado
            cursor.execute(
                """SELECT id, nombre, cantidad_stock, unidad_medida, precio_por_unidad
                   FROM productos WHERE id = %s FOR UPDATE""",
                (producto_id,)
            )
            producto = cursor.fetchone()
            if producto is None:
                raise ProductoNoEncontrado


            cantidad = Decimal(str(cantidad))
            disponible = Decimal(str(producto["cantidad_stock"]))
            if cantidad > disponible:
                raise StockInsuficiente(disponible)


            precio = Decimal(str(producto["precio_por_unidad"]))
            total = (cantidad * precio).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
            cursor.execute(
                "UPDATE productos SET cantidad_stock = cantidad_stock - %s WHERE id = %s",
                (cantidad, producto_id)
            )
            cursor.execute(
                """INSERT INTO ventas
```

```
                        (producto_id,    vendedor_id,    comprador_id,    producto_nombre,
cantidad,
                         unidad_medida, precio_por_unidad, total)
                        VALUES (%s, %s, %s, %s, %s, %s, %s, %s)""",
                    (producto_id,    vendedor_id,    comprador_id,    producto["nombre"],
cantidad,
                     producto["unidad_medida"], precio, total)
            )
            venta_id = cursor.lastrowid
            conn.commit()
            return {
                "id": venta_id,
                "producto_id": producto_id,
                "comprador_id": comprador_id,
                "producto_nombre": producto["nombre"],
                "cantidad": float(cantidad),
                "unidad_medida": producto["unidad_medida"],
                "precio_por_unidad": float(precio),
                "total": float(total),
            }
        except Exception:
            conn.rollback()
            raise
        finally:
            cursor.close()
            conn.close()
```

3.4 app.iniciar_sesion

```
@app.route("/api/sesion", methods=["POST"])
def iniciar_sesion():
    data = request.get_json(silent=True) or {}
    correo = data.get("correo", "")
    contrasena = data.get("contrasena", "")
    if not isinstance(correo, str) or not isinstance(contrasena, str):
        return jsonify({"error": "Correo y contraseña son obligatorios"}), 400


    usuario = usuarios.buscar_por_correo(correo.strip().lower())
    if (usuario is None or not usuario["activo"] or
            not check_password_hash(usuario["password_hash"], contrasena)):
        return jsonify({"error": "Correo o contraseña incorrectos"}), 401


    session.clear()
    session["usuario"] = datos_publicos(usuario)
    return jsonify(datos_publicos(usuario)), 200
```

3.5 app.requiere_rol

```
def requiere_rol(*roles):
    def decorador(funcion):
        @wraps(funcion)
        def protegido(*args, **kwargs):
            usuario = session.get("usuario")
            if usuario is None:
                return jsonify({"error": "Debes iniciar sesión"}), 401
            if roles and usuario["rol"] not in roles:
                return jsonify({"error": "No tienes permisos para esta operación"}), 403
            return funcion(*args, **kwargs)
        return protegido
    return decorador
```

4. Código de los Unit Tests

Las pruebas se agruparon en el archivo test_funciones_criticas.py. Cada función se aísla con dobles de prueba (fakes) cuando depende de la base de datos, y con el cliente de pruebas de Flask cuando depende del contexto web. En todos los casos el escenario se prepara en la sección marcada como Arrange, la acción se ejecuta en Act y la comparación se realiza en Assert.

4.1 Pruebas de validar_precio (5 casos)

```
def test_validar_precio_convierte_numero_y_redondea():
    # Arrange
    entrada = 10.999
    # Act
    resultado = validar_precio(entrada)
    # Assert
    assert resultado == 11.0


def test_validar_precio_acepta_string_numerico():
    # Arrange
    entrada = "1500.50"
    # Act
    resultado = validar_precio(entrada)
    # Assert
    assert resultado == 1500.50


def test_validar_precio_rechaza_cero_y_negativos():
    # Arrange
    entradas_invalidas = (0, -100, -0.01)
    # Act / Assert
    for valor in entradas_invalidas:
        with pytest.raises(ValueError):
            validar_precio(valor)


def test_validar_precio_rechaza_no_numericos_y_nulos():
    # Arrange
    entradas_invalidas = ("abc", None, float("inf"), float("nan"))
    # Act / Assert
    for valor in entradas_invalidas:
        with pytest.raises(ValueError):
            validar_precio(valor)


def test_validar_precio_respeta_el_techo_de_negocio():
    # Arrange
    valor_limite = 99999999.99
    valor_excedido = 100000000
    # Act
    resultado = validar_precio(valor_limite)
    # Assert
    assert resultado == 99999999.99
    with pytest.raises(ValueError):
        validar_precio(valor_excedido)
```

4.2 Pruebas de validar_cantidad (5 casos)

```
def test_validar_cantidad_positiva_redondea_a_tres_decimales():
    # Arrange
    entrada = "0.250"
    # Act
    resultado = validar_cantidad(entrada)
    # Assert
    assert resultado == 0.25



def test_validar_cantidad_cero_permitido_para_stock():
    # Arrange
    entrada = 0
    # Act
    resultado = validar_cantidad(entrada, permitir_cero=True)
    # Assert
    assert resultado == 0.0



def test_validar_cantidad_cero_no_permitido_en_venta():
    # Arrange
    entrada = 0
    # Act / Assert
    with pytest.raises(ValueError):
        validar_cantidad(entrada)



def test_validar_cantidad_rechaza_negativos_bool_y_nulos():
    # Arrange
    entradas_invalidas = (-1, True, None)
    # Act / Assert
    for valor in entradas_invalidas:
        with pytest.raises(ValueError):
            validar_cantidad(valor)



def test_validar_cantidad_rechaza_nan_y_exceso_del_tope():
    # Arrange
    nan = float("nan")
    excedido = 1000000000
    # Act / Assert
    with pytest.raises(ValueError):
        validar_cantidad(nan)
    with pytest.raises(ValueError):
        validar_cantidad(excedido)
```

4.3 Pruebas de VentaRepository.registrar (5 casos)

```
class CursorFake:
    def __init__(self, conexion):
        self.conexion = conexion
        self.resultado = None
        self.lastrowid = None


    def execute(self, consulta, parametros):
        sql = consulta.lstrip().upper()
        if sql.startswith("SELECT") and "FROM COMPRADORES" in sql:
            comprador_id = parametros[0]
            self.resultado = (
                {"id": comprador_id}
                if comprador_id in self.conexion.compradores
                else None
            )
```

```
        elif sql.startswith("SELECT") and "FROM PRODUCTOS" in sql:
            producto_id = parametros[0]
            producto = self.conexion.producto
            self.resultado = (
                dict(producto)
                if producto is not None and producto_id == producto["id"]
                else None
            )
        elif sql.startswith("UPDATE PRODUCTOS"):
            cantidad, producto_id = parametros
            assert producto_id == self.conexion.producto["id"]
            self.conexion.producto["cantidad_stock"] -= cantidad
        elif sql.startswith("INSERT INTO VENTAS"):
            self.conexion.ventas.append(parametros)
            self.lastrowid = len(self.conexion.ventas)

    def fetchone(self):
        return self.resultado


    def close(self):
        pass



class ConnectionFake:
    def __init__(self, producto, compradores=(12,)):
        self.producto = producto
        self.compradores = set(compradores)
        self.ventas = []
        self.commits = 0
        self.rollbacks = 0
        self.snapshot = None

    def cursor(self, dictionary=False):
        assert dictionary is True
        return CursorFake(self)


    def start_transaction(self):
        snapshot_producto = dict(self.producto) if self.producto else None
        self.snapshot = (snapshot_producto, list(self.ventas))


    def commit(self):
        self.commits += 1


    def rollback(self):
        self.rollbacks += 1
        self.producto, self.ventas = self.snapshot


    def close(self):
        pass



def producto_manzana():
    return {
        "id": 7,
        "nombre": "Manzana",
        "cantidad_stock": Decimal("4.250"),
        "unidad_medida": "kg",
        "precio_por_unidad": Decimal("2.35"),
    }



def test_venta_registrar_descuenta_stock_y_calcula_total(monkeypatch):
    # Arrange
    conexion = ConnectionFake(producto_manzana())
```

```
    monkeypatch.setattr(models, "get_connection", lambda: conexion)
    # Act
    venta = models.VentaRepository().registrar(7, 3, 1.25, 12)
    # Assert
    assert venta["total"] == 2.94
    assert venta["cantidad"] == 1.25
    assert venta["comprador_id"] == 12
    assert conexion.producto["cantidad_stock"] == Decimal("3.000")
    assert len(conexion.ventas) == 1
    assert conexion.commits == 1
    assert conexion.rollbacks == 0


def test_venta_registrar_usa_redondeo_half_up_del_total(monkeypatch):
    # Arrange
    producto = producto_manzana()
    producto["precio_por_unidad"] = Decimal("1.00")
    conexion = ConnectionFake(producto)
    monkeypatch.setattr(models, "get_connection", lambda: conexion)
    # Act
    venta = models.VentaRepository().registrar(7, 3, 1.005, 12)
    # Assert
    assert venta["total"] == 1.01
    assert conexion.commits == 1


def test_venta_registrar_stock_insuficiente_hace_rollback(monkeypatch):
    # Arrange
    conexion = ConnectionFake(producto_manzana())
    monkeypatch.setattr(models, "get_connection", lambda: conexion)
    # Act / Assert
    with pytest.raises(models.StockInsuficiente):
        models.VentaRepository().registrar(7, 3, 5, 12)
    assert conexion.producto["cantidad_stock"] == Decimal("4.250")
    assert conexion.ventas == []
    assert conexion.commits == 0
    assert conexion.rollbacks == 1


def test_venta_registrar_producto_inexistente_hace_rollback(monkeypatch):
    # Arrange
    conexion = ConnectionFake(producto=None)
    monkeypatch.setattr(models, "get_connection", lambda: conexion)
    # Act / Assert
    with pytest.raises(models.ProductoNoEncontrado):
        models.VentaRepository().registrar(999, 3, 1, 12)
    assert conexion.commits == 0
    assert conexion.rollbacks == 1


def test_venta_registrar_comprador_inexistente_hace_rollback(monkeypatch):
    # Arrange
    conexion = ConnectionFake(producto_manzana(), compradores=())
    monkeypatch.setattr(models, "get_connection", lambda: conexion)
    # Act / Assert
    with pytest.raises(models.CompradorNoEncontrado):
        models.VentaRepository().registrar(7, 3, 1, 999)
    assert conexion.ventas == []
    assert conexion.commits == 0
    assert conexion.rollbacks == 1
```

4.4 Pruebas de iniciar_sesion (5 casos)

```
class UsuariosFake:
    def __init__(self):
        self.usuarios = {
```

```
            "admin@test.local": {
                "id": 1,
                "nombre": "Admin",
                "correo": "admin@test.local",
                "password_hash": generate_password_hash("clave1234"),
                "rol": "admin",
                "activo": True,
            },
            "inactivo@test.local": {
                "id": 2,
                "nombre": "Bloqueado",
                "correo": "inactivo@test.local",
                "password_hash": generate_password_hash("clave1234"),
                "rol": "vendedor",
                "activo": False,
            },
        }

    def buscar_por_correo(self, correo):
        return self.usuarios.get(correo)



@pytest.fixture
def cliente_sesion(monkeypatch):
    monkeypatch.setattr(sistema, "usuarios", UsuariosFake())
    return sistema.app.test_client()



def test_iniciar_sesion_valido_crea_sesion(cliente_sesion):
    # Arrange
    credenciales = {"correo": "admin@test.local", "contrasena": "clave1234"}
    # Act
    respuesta = cliente_sesion.post("/api/sesion", json=credenciales)
    # Assert
    assert respuesta.status_code == 200
    assert respuesta.json["perfil"] == "Gerente"
    assert cliente_sesion.get("/api/sesion").status_code == 200



def test_iniciar_sesion_normaliza_correo(cliente_sesion):
    # Arrange
    credenciales = {"correo": "  ADMIN@Test.Local  ", "contrasena": "clave1234"}
    # Act
    respuesta = cliente_sesion.post("/api/sesion", json=credenciales)
    # Assert
    assert respuesta.status_code == 200
    assert respuesta.json["rol"] == "admin"



def test_iniciar_sesion_correo_o_contrasena_incorrectos(cliente_sesion):
    # Arrange
    inexistente = {"correo": "nadie@test.local", "contrasena": "clave1234"}
    mala_clave = {"correo": "admin@test.local", "contrasena": "incorrecta"}
    # Act / Assert
    assert cliente_sesion.post("/api/sesion", json=inexistente).status_code == 401
    assert cliente_sesion.post("/api/sesion", json=mala_clave).status_code == 401



def test_iniciar_sesion_usuario_inactivo_es_rechazado(cliente_sesion):
    # Arrange
    credenciales = {"correo": "inactivo@test.local", "contrasena": "clave1234"}
    # Act
    respuesta = cliente_sesion.post("/api/sesion", json=credenciales)
    # Assert
    assert respuesta.status_code == 401
```

```
def test_iniciar_sesion_campos_no_texto_devuelve_400(cliente_sesion):
    # Arrange
    credenciales = {"correo": "admin@test.local", "contrasena": 12345678}
    # Act
    respuesta = cliente_sesion.post("/api/sesion", json=credenciales)
    # Assert
    assert respuesta.status_code == 400
```

4.5 Pruebas de requiere_rol (5 casos)

```
def app_con_permisos():
    aplicacion = Flask(__name__)
    aplicacion.secret_key = "test-secreto"

    @aplicacion.route("/solo-admin")
    @sistema.requiere_rol("admin")
    def solo_admin():
        return jsonify({"ok": True})

    @aplicacion.route("/admin-o-vendedor")
    @sistema.requiere_rol("admin", "vendedor")
    def admin_o_vendedor():
        return jsonify({"ok": True})

    @aplicacion.route("/cualquier-rol")
    @sistema.requiere_rol()
    def cualquier_rol():
        return jsonify({"ok": True})

    @aplicacion.route("/entrar/<rol>")
    def entrar(rol):
        session["usuario"] = {
            "id": 1,
            "nombre": rol,
            "correo": f"{rol}@test.local",
            "rol": rol,
            "perfil": rol,
        }
        return jsonify({"ok": True})

    return aplicacion


@pytest.fixture
def cliente_permisos():
    return app_con_permisos().test_client()


def test_requiere_rol_sin_sesion_devuelve_401(cliente_permisos):
    # Arrange
    destino = "/solo-admin"
    # Act
    respuesta = cliente_permisos.get(destino)
    # Assert
    assert respuesta.status_code == 401
    assert respuesta.json["error"] == "Debes iniciar sesión"


def test_requiere_rol_deniega_rol_no_autorizado(cliente_permisos):
    # Arrange
    cliente_permisos.get("/entrar/vendedor")
```

```
    # Act
    respuesta = cliente_permisos.get("/solo-admin")
    # Assert
    assert respuesta.status_code == 403


def test_requiere_rol_permite_rol_autorizado(cliente_permisos):
    # Arrange
    cliente_permisos.get("/entrar/admin")
    # Act
    respuesta = cliente_permisos.get("/solo-admin")
    # Assert
    assert respuesta.status_code == 200
    assert respuesta.json["ok"] is True


def test_requiere_rol_admite_multiples_roles(cliente_permisos):
    # Arrange
    cliente_permisos.get("/entrar/vendedor")
    # Act
    respuesta = cliente_permisos.get("/admin-o-vendedor")
    # Assert
    assert respuesta.status_code == 200


def test_requiere_rol_sin_argumentos_solo_exige_sesion(cliente_permisos):
    # Arrange
    cliente_permisos.get("/entrar/compras")
    # Act
    respuesta = cliente_permisos.get("/cualquier-rol")
    # Assert
    assert respuesta.status_code == 200
```

5. Casos de prueba: Arrange, Act y Assert

Para cada función se identificaron cinco casos de prueba. La columna Arrange describe la preparación de la entrada o del estado; la columna Act indica la acción ejecutada; la columna Assert muestra la respuesta esperada y comparada por la prueba.

5.1 Casos de validar_precio

Tabla 2. Casos de prueba de validar_precio.

Caso de prueba	Arrange (entrada / estado)	Act (acción)	Assert (resultado esperado)	Estado
1	entrada = 10.999	validar_precio(10.999)	devuelve 11.0	PASA
2	entrada = "1500.50"	validar_precio("1500.50")	devuelve 1500.50	PASA
3	entrada ∈ {0, −100, −0.01}	validar_precio(valor)	lanza ValueError	PASA
4	entrada ∈ {"abc", None, inf, nan}	validar_precio(valor)	lanza ValueError	PASA

5	entrada = 99999999.99 y 100000000	validar_precio(...)	acepta el límite y rechaza el exceso	PASA

Estado: PASA = la aserción se cumplió en la ejecución real de pytest.

5.2 Casos de validar_cantidad

Tabla 3. Casos de prueba de validar_cantidad.

Caso de prueba	Arrange (entrada / estado)	Act (acción)	Assert (resultado esperado)	Estado
1	entrada = "0.250"	validar_cantidad("0.250")	devuelve 0.25	PASA
2	entrada = 0 (permitir_cero=True)	validar_cantidad(0, permitir_cero=True)	devuelve 0.0	PASA
3	entrada = 0 (permitir_cero=False)	validar_cantidad(0)	lanza ValueError	PASA
4	entrada ∈ {−1, True, None}	validar_cantidad(valor)	lanza ValueError	PASA
5	entrada = nan y 1000000000	validar_cantidad(valor)	lanza ValueError	PASA

Estado: PASA = la aserción se cumplió en la ejecución real de pytest.

5.3 Casos de VentaRepository.registrar

Tabla 4. Casos de prueba de VentaRepository.registrar.

Caso de prueba	Arrange (entrada / estado)	Act (acción)	Assert (resultado esperado)	Estado
1	stock 4.250 kg, precio 2.35, venta 1.25	registrar(7, 3, 1.25, 12)	total 2.94; stock 3.000; 1 commit y 0 rollback	PASA
2	precio 1.00, cantidad 1.005	registrar(7, 3, 1.005, 12)	total 1.01 (redondeo HALF_UP)	PASA
3	stock 4.250 kg, venta 5	registrar(7, 3, 5, 12)	lanza StockInsuficiente; stock intacto; 1 rollback	PASA
4	producto inexistente (id 999)	registrar(999, 3, 1, 12)	lanza ProductoNoEncontrado; 1 rollback	PASA
5	comprador inexistente (id 999)	registrar(7, 3, 1, 999)	lanza CompradorNoEncontrado; 1 rollback	PASA

Estado: PASA = la aserción se cumplió en la ejecución real de pytest.

5.4 Casos de iniciar_sesion

Tabla 5. Casos de prueba de iniciar_sesion.

Caso de prueba	Arrange (entrada / estado)	Act (acción)	Assert (resultado esperado)	Estado
1	usuario admin activo	POST /api/sesion	200; perfil "Gerente"; sesión creada	PASA
2	correo " ADMIN@Test.Local "	POST /api/sesion	200; rol admin (correo normalizado)	PASA
3	correo inexistente / clave errónea	POST /api/sesion	401 en ambos casos	PASA
4	usuario con activo = False	POST /api/sesion	401	PASA
5	contrasena = 12345678 (no texto)	POST /api/sesion	400	PASA

Estado: PASA = la aserción se cumplió en la ejecución real de pytest.

5.5 Casos de requiere_rol

Tabla 6. Casos de prueba de requiere_rol.

Caso de prueba	Arrange (entrada / estado)	Act (acción)	Assert (resultado esperado)	Estado
1	sin sesión iniciada	GET /solo-admin	401 "Debes iniciar sesión"	PASA
2	sesión con rol vendedor	GET /solo-admin	403	PASA
3	sesión con rol admin	GET /solo-admin	200	PASA
4	sesión con rol vendedor	GET /admin-o-vendedor	200	PASA
5	sesión con rol compras	GET /cualquier-rol	200	PASA

Estado: PASA = la aserción se cumplió en la ejecución real de pytest.

6. Evidencia de ejecución

Las siguientes capturas corresponden a la ejecución real de la suite de pruebas con pytest. En total se ejecutaron 25 casos y todos resultaron exitosos (25 passed).

```
Unit Tests - Suite completa (25 casos)

============================================= test session starts ==============================================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0 -- /tmp/opencode/venv_tarea1/bin/python
cachedir: .pytest_cache
rootdir: /home/eynor/Documents/semestr7/Testing /hito3/tarea_1
plugins: anyio-4.15.1, platformdirs-4.13.0
collecting ... collected 25 items

test_funciones_criticas.py::test_validar_precio_convierte_numero_y_redondea PASSED                    [  4%]
test_funciones_criticas.py::test_validar_precio_acepta_string_numerico PASSED                         [  8%]
test_funciones_criticas.py::test_validar_precio_rechaza_cero_y_negativos PASSED                       [ 12%]
test_funciones_criticas.py::test_validar_precio_rechaza_no_numericos_y_nulos PASSED                   [ 16%]
test_funciones_criticas.py::test_validar_precio_respeta_el_techo_de_negocio PASSED                    [ 20%]
test_funciones_criticas.py::test_validar_cantidad_positiva_redondea_a_tres_decimales PASSED           [ 24%]
test_funciones_criticas.py::test_validar_cantidad_cero_permitido_para_stock PASSED                    [ 28%]
test_funciones_criticas.py::test_validar_cantidad_cero_no_permitido_en_venta PASSED                   [ 32%]
test_funciones_criticas.py::test_validar_cantidad_rechaza_negativos_bool_y_nulos PASSED               [ 36%]
test_funciones_criticas.py::test_validar_cantidad_rechaza_nan_y_exceso_del_tope PASSED                [ 40%]
test_funciones_criticas.py::test_venta_registrar_descuenta_stock_y_calcula_total PASSED               [ 44%]
test_funciones_criticas.py::test_venta_registrar_usa_redondeo_half_up_del_total PASSED                [ 48%]
test_funciones_criticas.py::test_venta_registrar_stock_insuficiente_hace_rollback PASSED              [ 52%]
test_funciones_criticas.py::test_venta_registrar_producto_inexistente_hace_rollback PASSED            [ 56%]
test_funciones_criticas.py::test_venta_registrar_comprador_inexistente_hace_rollback PASSED           [ 60%]
test_funciones_criticas.py::test_iniciar_sesion_valido_crea_sesion PASSED                             [ 64%]
test_funciones_criticas.py::test_iniciar_sesion_normaliza_correo PASSED                               [ 68%]
test_funciones_criticas.py::test_iniciar_sesion_correo_o_contrasena_incorrectos PASSED                [ 72%]
test_funciones_criticas.py::test_iniciar_sesion_usuario_inactivo_es_rechazado PASSED                  [ 76%]
test_funciones_criticas.py::test_iniciar_sesion_campos_no_texto_devuelve_400 PASSED                   [ 80%]
test_funciones_criticas.py::test_requiere_rol_sin_sesion_devuelve_401 PASSED                          [ 84%]
test_funciones_criticas.py::test_requiere_rol_deniega_rol_no_autorizado PASSED                        [ 88%]
test_funciones_criticas.py::test_requiere_rol_permite_rol_autorizado PASSED                           [ 92%]
test_funciones_criticas.py::test_requiere_rol_admite_multiples_roles PASSED                           [ 96%]
test_funciones_criticas.py::test_requiere_rol_sin_argumentos_solo_exige_sesion PASSED                 [100%]

============================================== 25 passed in 2.88s ==============================================
```

Figura 1. Ejecución completa de la suite: 25 casos exitosos.

```
validar_precio() - 5 casos

============================================= test session starts ==============================================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0 -- /tmp/opencode/venv_tarea1/bin/python
cachedir: .pytest_cache
rootdir: /home/eynor/Documents/semestr7/Testing /hito3/tarea_1
plugins: anyio-4.15.1, platformdirs-4.13.0
collecting ... collected 25 items / 20 deselected / 5 selected

test_funciones_criticas.py::test_validar_precio_convierte_numero_y_redondea PASSED                    [ 20%]
test_funciones_criticas.py::test_validar_precio_acepta_string_numerico PASSED                         [ 40%]
test_funciones_criticas.py::test_validar_precio_rechaza_cero_y_negativos PASSED                       [ 60%]
test_funciones_criticas.py::test_validar_precio_rechaza_no_numericos_y_nulos PASSED                   [ 80%]
test_funciones_criticas.py::test_validar_precio_respeta_el_techo_de_negocio PASSED                    [100%]

========================================== 5 passed, 20 deselected in 0.86s ==========================================
```

Figura 2. Pruebas de validar_precio: 5 casos exitosos.

```
validar_cantidad() - 5 casos

============================================= test session starts ==============================================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0 -- /tmp/opencode/venv_tarea1/bin/python
cachedir: .pytest_cache
rootdir: /home/eynor/Documents/semestr7/Testing /hito3/tarea_1
plugins: anyio-4.15.1, platformdirs-4.13.0
collecting ... collected 25 items / 20 deselected / 5 selected

test_funciones_criticas.py::test_validar_cantidad_positiva_redondea_a_tres_decimales  PASSED                    [ 20%]
test_funciones_criticas.py::test_validar_cantidad_cero_permitido_para_stock  PASSED                            [ 40%]
test_funciones_criticas.py::test_validar_cantidad_cero_no_permitido_en_venta  PASSED                          [ 60%]
test_funciones_criticas.py::test_validar_cantidad_rechaza_negativos_bool_y_nulos  PASSED                      [ 80%]
test_funciones_criticas.py::test_validar_cantidad_rechaza_nan_y_exceso_del_tope  PASSED                       [100%]

========================================= 5 passed, 20 deselected in 0.77s =========================================
```

Figura 3. Pruebas de validar_cantidad: 5 casos exitosos.

```
VentaRepository.registrar() - 5 casos

============================================= test session starts ==============================================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0 -- /tmp/opencode/venv_tarea1/bin/python
cachedir: .pytest_cache
rootdir: /home/eynor/Documents/semestr7/Testing /hito3/tarea_1
plugins: anyio-4.15.1, platformdirs-4.13.0
collecting ... collected 25 items / 20 deselected / 5 selected

test_funciones_criticas.py::test_venta_registrar_descuenta_stock_y_calcula_total  PASSED                       [ 20%]
test_funciones_criticas.py::test_venta_registrar_usa_redondeo_half_up_del_total  PASSED                       [ 40%]
test_funciones_criticas.py::test_venta_registrar_stock_insuficiente_hace_rollback  PASSED                     [ 60%]
test_funciones_criticas.py::test_venta_registrar_producto_inexistente_hace_rollback  PASSED                   [ 80%]
test_funciones_criticas.py::test_venta_registrar_comprador_inexistente_hace_rollback  PASSED                  [100%]

========================================= 5 passed, 20 deselected in 0.62s =========================================
```

Figura 4. Pruebas de VentaRepository.registrar: 5 casos exitosos.

```
app.iniciar_sesion() - 5 casos

============================================= test session starts ==============================================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0 -- /tmp/opencode/venv_tarea1/bin/python
cachedir: .pytest_cache
rootdir: /home/eynor/Documents/semestr7/Testing /hito3/tarea_1
plugins: anyio-4.15.1, platformdirs-4.13.0
collecting ... collected 25 items / 20 deselected / 5 selected

test_funciones_criticas.py::test_iniciar_sesion_valido_crea_sesion  PASSED                                   [ 20%]
test_funciones_criticas.py::test_iniciar_sesion_normaliza_correo  PASSED                                     [ 40%]
test_funciones_criticas.py::test_iniciar_sesion_correo_o_contrasena_incorrectos  PASSED                      [ 60%]
test_funciones_criticas.py::test_iniciar_sesion_usuario_inactivo_es_rechazado  PASSED                        [ 80%]
test_funciones_criticas.py::test_iniciar_sesion_campos_no_texto_devuelve_400  PASSED                         [100%]

========================================= 5 passed, 20 deselected in 2.89s =========================================
```

Figura 5. Pruebas de iniciar_sesion: 5 casos exitosos.

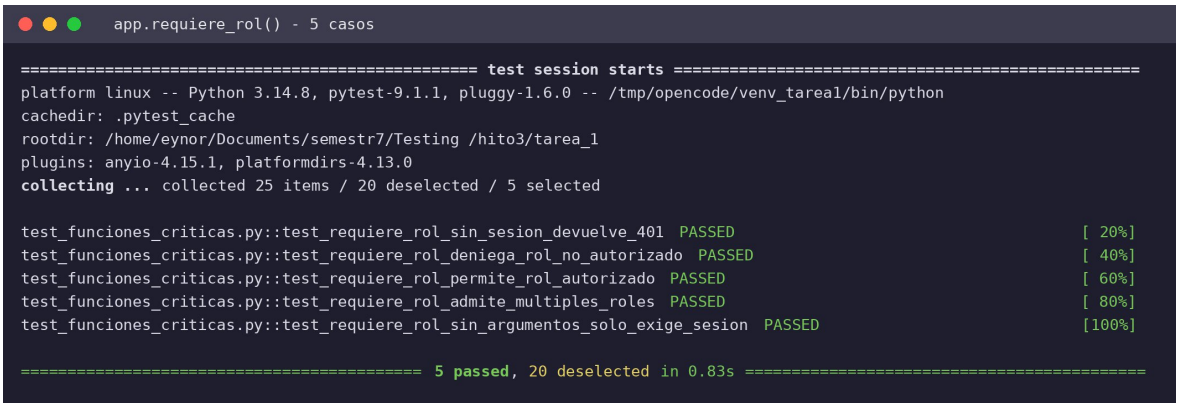


Figura 6. Pruebas de requiere_rol: 5 casos exitosos.

7. Conclusiones

- Las cinco funciones seleccionadas cubren el negocio transaccional, la entrada del dinero, el control de inventario y la seguridad de acceso, que son los puntos donde un error produce consecuencias reales.
- El patrón Arrange–Act–Assert permitió separar con claridad la preparación del escenario, la acción y la verificación, lo que hace que cada caso sea legible y repetible.
- El uso de dobles de prueba y de monkeypatch aisló por completo la base de datos, de modo que las pruebas unitarias se ejecutan de forma rápida y determinista.
- Los 25 casos se ejecutaron sin fallos, lo que confirma el comportamiento correcto de las funciones ante entradas válidas, entradas de frontera y situaciones de error.

BIBLIOGRAFÍA

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

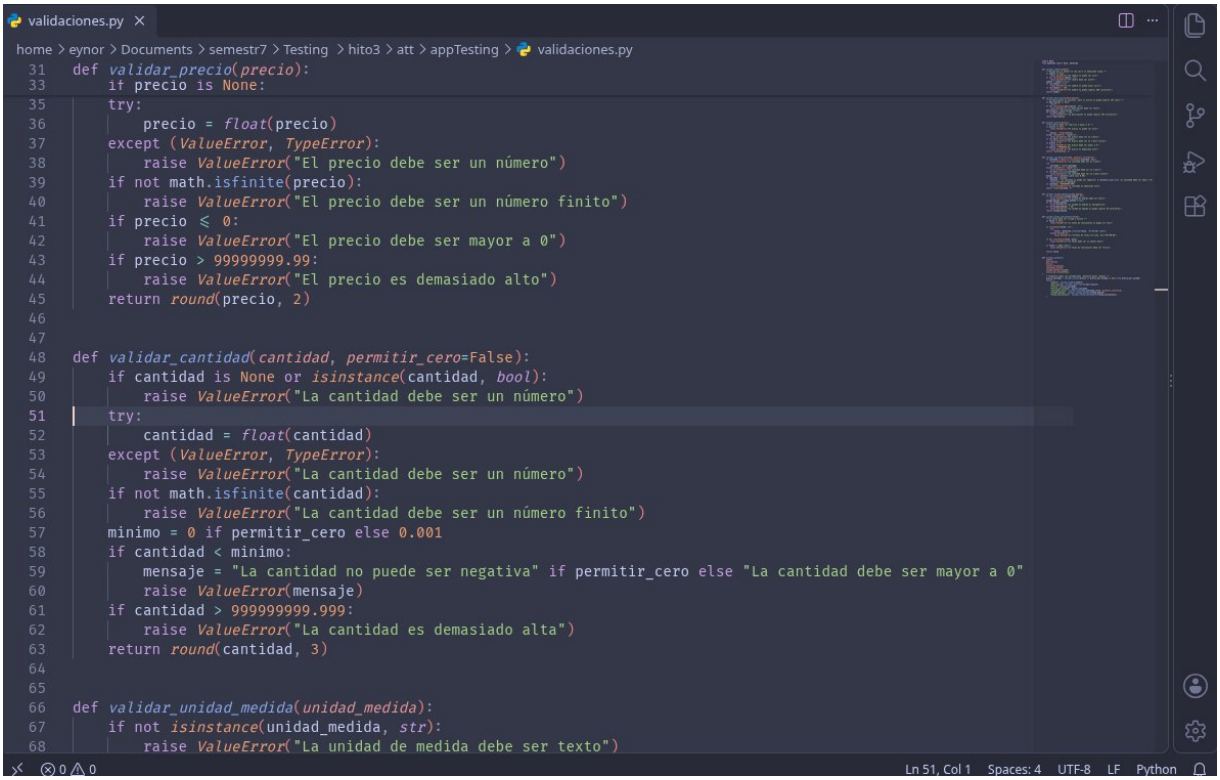
Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2.ª ed.). O'Reilly Media.

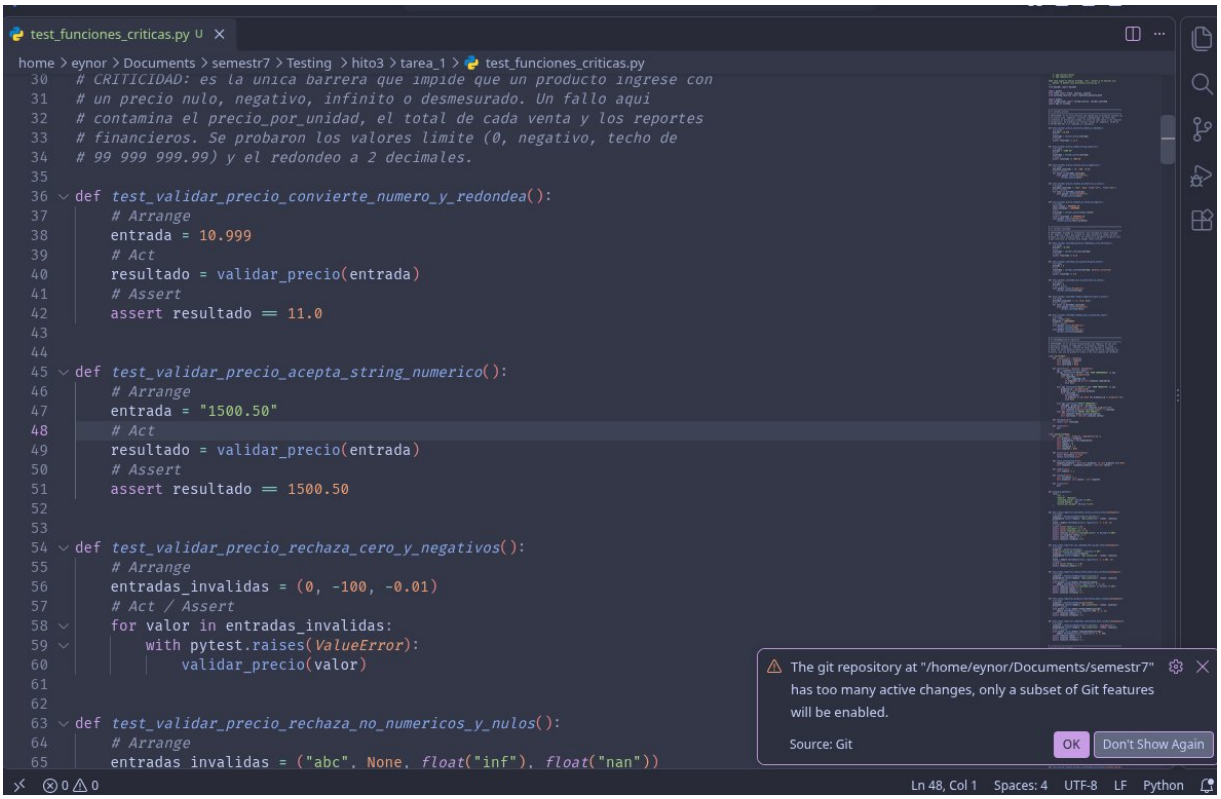
Kaner, C., Falk, J. y Nguyen, H. Q. (1999). Testing Computer Software (2.ª ed.). Wiley.

Pressman, R. S. (2014). Ingeniería del software: un enfoque práctico (9.ª ed.). McGraw-Hill.

pytest. (2024). pytest documentation. https://docs.pytest.org

ANEXO





test_funciones_criticas.py U

home > eynor > Documents > semestr7 > Testing > hito3 > tarea_1 > test_funciones_criticas.py

```python
# CRITICIDAD: es la unica barrera que impide que un producto ingrese con
# un precio nulo, negativo, infinito o desmesurado. Un fallo aqui
# contamina el precio_por_unidad, el total de cada venta y los reportes
# financieros. Se probaron los valores limite (0, negativo, techo de
# 99 999 999.99) y el redondeo a 2 decimales.

def test_validar_precio_convierte_numero_y_redondea():
    # Arrange
    entrada = 10.999
    # Act
    resultado = validar_precio(entrada)
    # Assert
    assert resultado == 11.0


def test_validar_precio_acepta_string_numerico():
    # Arrange
    entrada = "1500.50"
    # Act
    resultado = validar_precio(entrada)
    # Assert
    assert resultado == 1500.50


def test_validar_precio_rechaza_cero_y_negativos():
    # Arrange
    entradas_invalidas = (0, -100, -0.01)
    # Act / Assert
    for valor in entradas_invalidas:
        with pytest.raises(ValueError):
            validar_precio(valor)

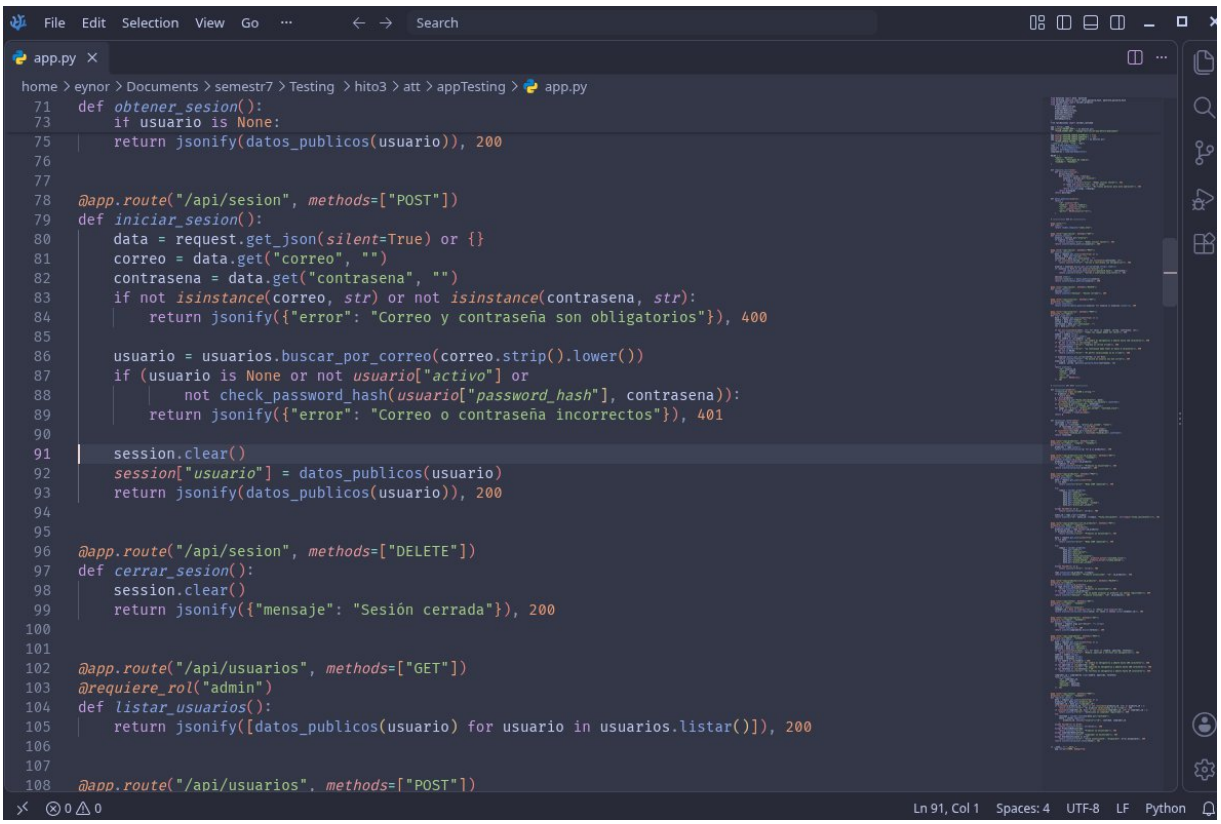

def test_validar_precio_rechaza_no_numericos_y_nulos():
    # Arrange
    entradas_invalidas = ("abc", None, float("inf"), float("nan"))
```

The git repository at "/home/eynor/Documents/semestr7" has too many active changes, only a subset of Git features will be enabled.

Source: Git

OK | Don't Show Again

Ln 48, Col 1 | Spaces: 4 | UTF-8 | LF | Python



File Edit Selection View Go ... Search

app.py

home > eynor > Documents > semestr7 > Testing > hito3 > att > appTesting > app.py

```python
def obtener_sesion():
    if usuario is None:
    return jsonify(datos_publicos(usuario)), 200


@app.route("/api/sesion", methods=["POST"])
def iniciar_sesion():
    data = request.get_json(silent=True) or {}
    correo = data.get("correo", "")
    contrasena = data.get("contrasena", "")
    if not isinstance(correo, str) or not isinstance(contrasena, str):
        return jsonify({"error": "Correo y contraseña son obligatorios"}), 400

    usuario = usuarios.buscar_por_correo(correo.strip().lower())
    if (usuario is None or not usuario["activo"] or
            not check_password_hash(usuario["password_hash"], contrasena)):
        return jsonify({"error": "Correo o contraseña incorrectos"}), 401

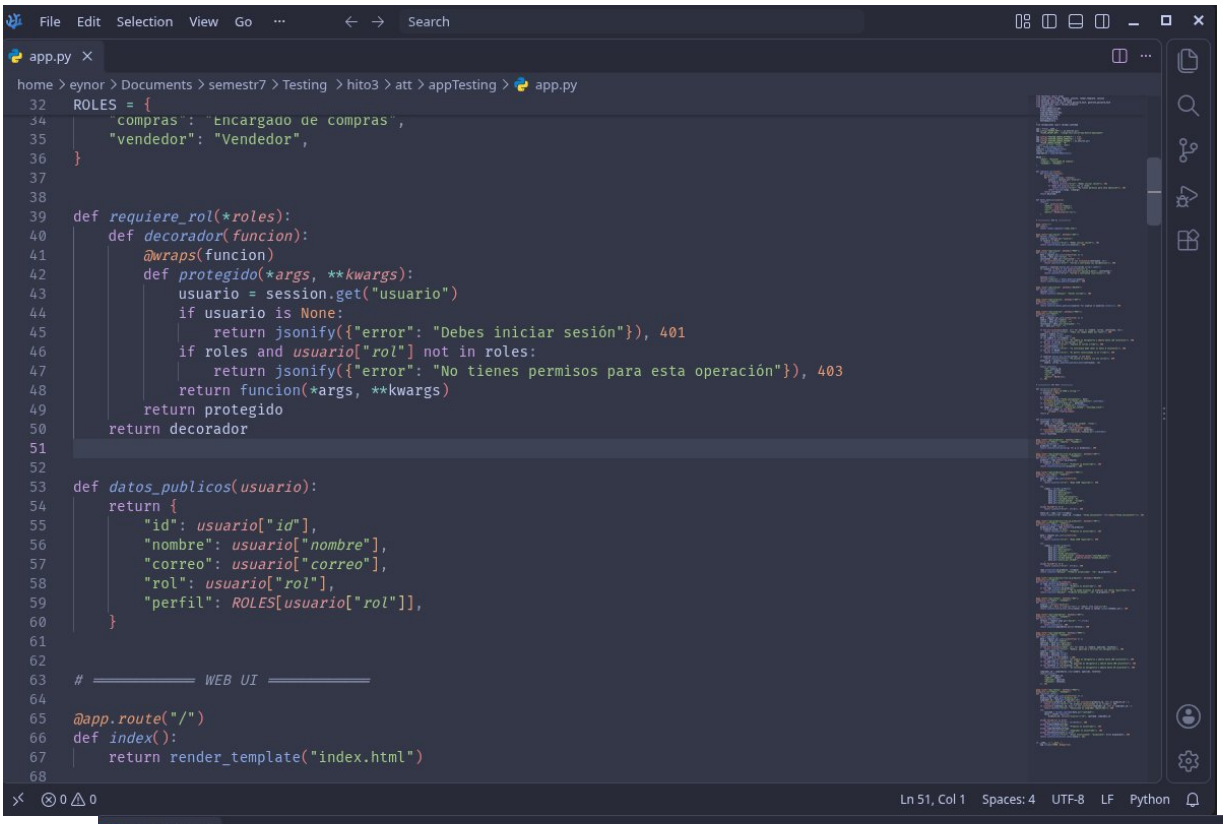
    session.clear()
    session["usuario"] = datos_publicos(usuario)
    return jsonify(datos_publicos(usuario)), 200


@app.route("/api/sesion", methods=["DELETE"])
def cerrar_sesion():
    session.clear()
    return jsonify({"mensaje": "Sesión cerrada"}), 200


@app.route("/api/usuarios", methods=["GET"])
@requiere_rol("admin")
def listar_usuarios():
    return jsonify([datos_publicos(usuario) for usuario in usuarios.listar()]), 200


@app.route("/api/usuarios", methods=["POST"])
```

Ln 91, Col 1 | Spaces: 4 | UTF-8 | LF | Python



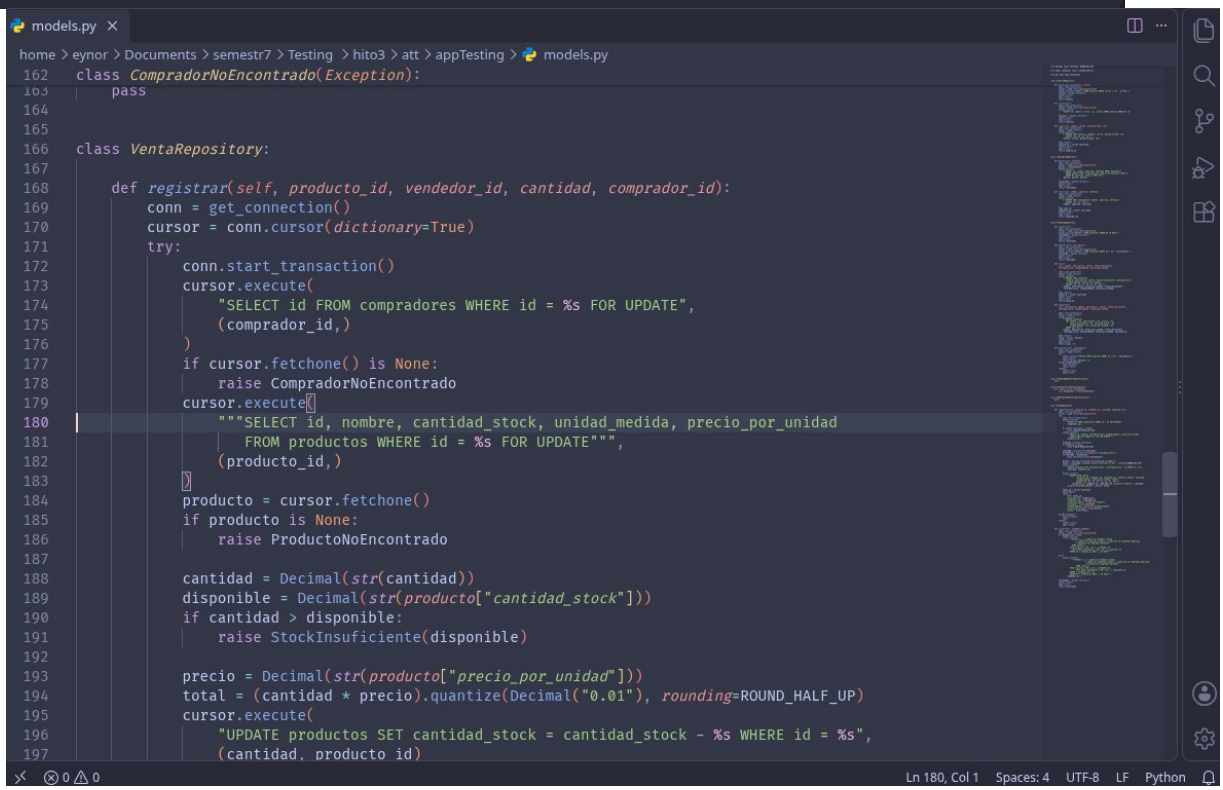
```python
ROLES = {
    "compras": "Encargado de compras",
    "vendedor": "Vendedor",
}


def requiere_rol(*roles):
    def decorador(funcion):
        @wraps(funcion)
        def protegido(*args, **kwargs):
            usuario = session.get("usuario")
            if usuario is None:
                return jsonify({"error": "Debes iniciar sesión"}), 401
            if roles and usuario["rol"] not in roles:
                return jsonify({"error": "No tienes permisos para esta operación"}), 403
            return funcion(*args, **kwargs)
        return protegido
    return decorador


def datos_publicos(usuario):
    return {
        "id": usuario["id"],
        "nombre": usuario["nombre"],
        "correo": usuario["correo"],
        "rol": usuario["rol"],
        "perfil": ROLES[usuario["rol"]],
    }


# ========== WEB UI ==========

@app.route("/")
def index():
    return render_template("index.html")
```



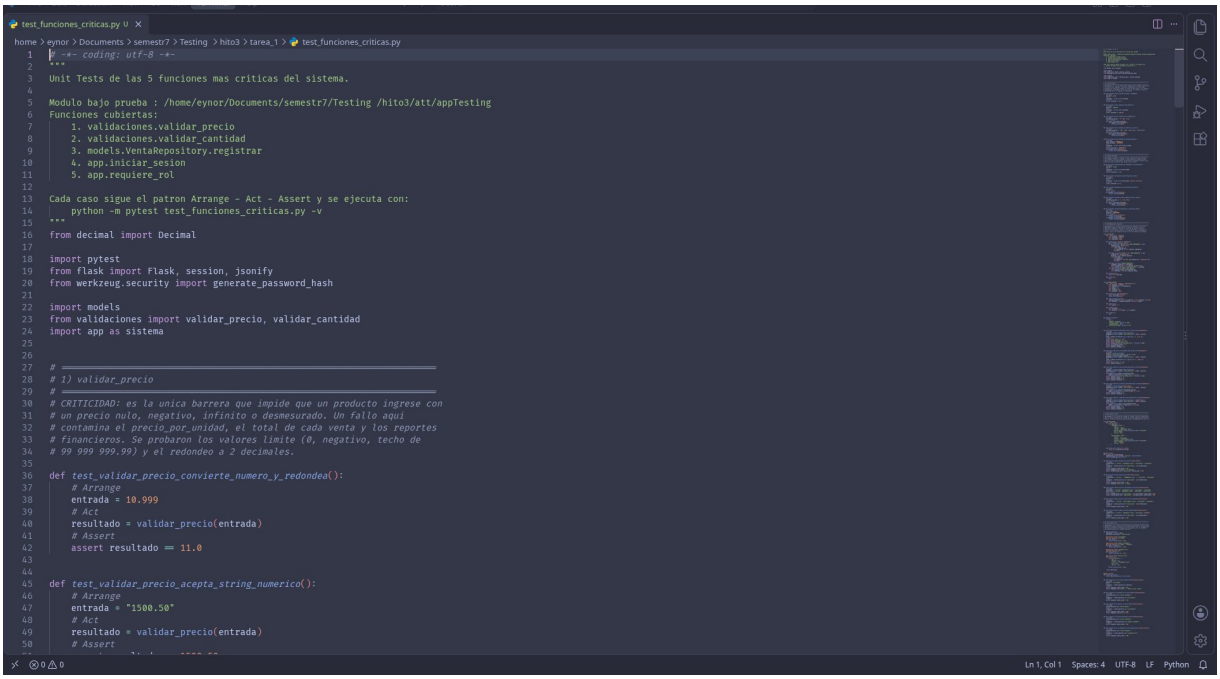
```python
class CompradorNoEncontrado(Exception):
    pass


class VentaRepository:

    def registrar(self, producto_id, vendedor_id, cantidad, comprador_id):
        conn = get_connection()
        cursor = conn.cursor(dictionary=True)
        try:
            conn.start_transaction()
            cursor.execute(
                "SELECT id FROM compradores WHERE id = %s FOR UPDATE",
                (comprador_id,)
            )
            if cursor.fetchone() is None:
                raise CompradorNoEncontrado
            cursor.execute(
                """SELECT id, nombre, cantidad_stock, unidad_medida, precio_por_unidad
                   FROM productos WHERE id = %s FOR UPDATE""",
                (producto_id,)
            )
            producto = cursor.fetchone()
            if producto is None:
                raise ProductoNoEncontrado

            cantidad = Decimal(str(cantidad))
            disponible = Decimal(str(producto["cantidad_stock"]))
            if cantidad > disponible:
                raise StockInsuficiente(disponible)

            precio = Decimal(str(producto["precio_por_unidad"]))
            total = (cantidad * precio).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
            cursor.execute(
                "UPDATE productos SET cantidad_stock = cantidad_stock - %s WHERE id = %s",
                (cantidad, producto_id)
```


test_funciones_criticas.py U
home > eynor > Documents > semestr7 > Testing > hito3 > tarea_1 > test_funciones_criticas.py
-*- coding: utf-8 -*-
"""
Unit Tests de las 5 funciones mas criticas del sistema.
Modulo bajo prueba : /home/eynor/Documents/semestr7/Testing /hito3/att/appTesting
Funciones cubiertas:
1. validaciones.validar_precio
2. validaciones.validar_cantidad
3. models.VentaRepository.registrar
4. app.iniciar_sesion
5. app.requiere_rol
Cada caso sigue el patron Arrange - Act - Assert y se ejecuta con:
python -m pytest test_funciones_criticas.py -v
"""
from decimal import Decimal
import pytest
from flask import Flask, session, jsonify
from werkzeug.security import generate_password_hash
import models
from validaciones import validar_precio, validar_cantidad
import app as sistema
═══
1) validar_precio
═══
CRITICIDAD: es la unica barrera que impide que un producto ingrese con
un precio nulo, negativo, infinito o desmesurado. Un fallo aqui
contamina el precio_por_unidad, el total de cada venta y los reportes
financieros. Se probaron los valores limite (0, negativo, techo de
99 999 999.99) y el redondeo a 2 decimales.
def test_validar_precio_convierte_numero_y_redondea():
Arrange
entrada = 10.999
Act
resultado = validar_precio(entrada)
Assert
assert resultado == 11.0
def test_validar_precio_acepta_string_numerico():
Arrange
entrada = "1500.50"
Act
resultado = validar_precio(entrada)
Assert
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python